

Customer Churn Analysis

Identifying High-Risk Customers and Churn Drivers

Project Objective: The objective of this project is to analyze customer churn by combining customer, subscription, and support data. The analysis aims to identify churn patterns, high-risk customer segments, factors associated with churn, and potential opportunities to improve customer retention.

Tools Used: Python, Pandas, NumPy, SQLite, Matplotlib, and Seaborn.

1. Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sqlite3
```

2. Data Loading

```
conn = sqlite3.connect('customer_churn.db')

sql_query = """
    SELECT name
    FROM sqlite_master
    WHERE type='table'
"""

tables = pd.read_sql(sql_query, conn)

#create dataframe for each table
for table_name in tables['name']:
    df = pd.read_sql(f"SELECT * FROM {table_name}", conn)
    globals()[f"df_{table_name}"] = df
    print(f"Created dataframe: df_{table_name}")

conn.close()
```

```
Created dataframe: df_db_customer
Created dataframe: df_db_subscription
Created dataframe: df_db_support
```

3. Data Cleaning and Preparation

```
#df_db_customer.head()
df_db_customer.tail()
```

	customerid	name	country	state	gender	dob	interests	pincode
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	None
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	None
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	None
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	None
20	0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	None

```
df_db_customer.info()

<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   customerid  21 non-null    str
1   name        21 non-null    str
2   country     18 non-null    str
3   state       21 non-null    str
4   gender      21 non-null    str
5   dob         21 non-null    str
6   interests   4 non-null     str
7   pincode     0 non-null     object
dtypes: object(1), str(7)
memory usage: 2.5+ KB
```

```
# a. drop columns: interest and pincode
# rename col: name
# change data type: dob
# data standardization: gender
# fix missing values: country
```

```
# rename col: name

df_db_customer.rename(columns = {'name' : 'customer_name'}, inplace = True)
```

```
# drop columns: interest and pincode

df_db_customer.drop(df_db_customer.columns[-2:], axis=1, inplace = True)
```

```
df_db_customer
```

	customerid	customer_name	country	state	gender	dob
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00
5	0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10 00:00:00
6	0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00
7	0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00
8	0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07 00:00:00
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00
10	0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22 00:00:00
11	0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18 00:00:00
12	0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01 00:00:00
13	0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25 00:00:00
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11 00:00:00
15	0020-INWCK	parvati	India	Delhi	Female	1986-02-28 00:00:00
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00
20	0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00

change data type: dob

df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])

data standardization: gender

#df_db_customer['gender'].unique()
df_db_customer['gender'] = df_db_customer['gender'].replace({'Men': 'Male', 'Women': 'Female'})

fix missing values: country

df_db_customer[df_db_customer['country'].isna()]

	customerid	customer_name	country	state	gender	dob
5	0013-MHZWF	durga	NaN	Delhi	Female	1988-12-10
8	0015-UOCOJ	maya	NaN	Kathmandu	Female	1985-07-07
12	0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01

state_country_mapping = df_db_customer.dropna(subset=['country']).set_index('state')['country'].to_dict()
df_db_customer['country'] = df_db_customer['country'].fillna(df_db_customer['state'].map(state_country_mapping))

```
df_db_customer[df_db_customer['country'].isna()]
```

customerid	customer_name	country	state	gender	dob
------------	---------------	---------	-------	--------	-----

```
df_db_subscription.head()
```

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract_type	cancellation_date	canc
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaN	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaN	
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaN	
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	



```
df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerid            21 non-null    str
1   subscription_start_date 21 non-null    str
2   subscription_type       21 non-null    str
3   renewal_date           21 non-null    str
4   plan_type              21 non-null    str
5   contract_type          21 non-null    str
6   cancellation_date       6 non-null     str
7   cancellation_reason     6 non-null     str
8   monthly_charges        21 non-null    float64
9   cltv                   21 non-null    int64
10  churn_score            21 non-null    int64
dtypes: float64(1), int64(2), str(8)
memory usage: 3.1 KB
```

```
date_col = ['subscription_start_date', 'renewal_date', 'cancellation_date']

df_db_subscription[date_col] = df_db_subscription[date_col].apply(pd.to_datetime)
```

```
df_db_support.head()
```

	customerid	complaint_date	escalations	csat_score	complaint_count
2	0013-EXCHZ	2024-01-20	Y	20	1
5	0017-IUDMW	2024-04-10	Y	25	1
1	0003-MKNFE	2024-08-28	Y	10	2
8	0022-TCJCI	2024-09-14	N	90	2
6	0019-EFAEP	2024-09-27	Y	30	1

```
df_db_support.info()
```

```
df_db_support['complaint_date'] = pd.to_datetime(df_db_support['complaint_date'])
```

```
df_db_support.drop(columns=['col_1', 'comment'], inplace = True )
```

4. Feature Engineering

```
# create a new col using existing col: churn flag
df_db_subscription['churn_flag'] = np.where(df_db_subscription['cancellation_date'].notna(), 1, 0)
```

```
df_db_subscription.head()
```

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract_type	cancellation_date	canc
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	

5. Data Integration

```
# first fix support table duplicate then merge
df = (df_db_subscription
      .merge(df_db_customer, on = 'customerid', how= 'left')
      .merge(df_db_support, on = 'customerid', how= 'left'))
```

```
df.shape
```

(23, 20)

```
df_db_support['complaint_count'] = df_db_support.groupby('customerid')['customerid'].transform('count')
```

```
df_db_support = df_db_support.sort_values('complaint_date').drop_duplicates('customerid', keep= 'last')
```

```
# merge
df = (df_db_subscription
      .merge(df_db_customer, on = 'customerid', how= 'left')
      .merge(df_db_support, on = 'customerid', how= 'left'))
```

```
df.shape
```

(21, 21)

```
df.to_csv('exported_churn_data.csv', index=False)
```

6. Exploratory Data Analysis and KPI Calculation

```
# churn rate
```

```
df.columns
```

```
Index(['customerid', 'subscription_start_date', 'subscription_type',  
      'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',  
      'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',  
      'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',  
      'complaint_date', 'escalations', 'csat_score', 'complaint_count'],  
      dtype='str')
```

```
churn_rate = df['churn_flag'].mean() * 100  
print("Churn Rate =", round(churn_rate, 2), "%")
```

Churn Rate = 28.57 %

```
# Retention Rate  
retention_rate = 100-churn_rate  
print("Retention Rate =", round(retention_rate, 2), "%")
```

Retention Rate = 71.43 %

```
# churn by plan type  
churn_by_plan = (df.groupby('plan_type')['churn_flag'].mean().mul(100).round(2).reset_index(name='churn_rate_p  
print(churn_by_plan)
```

	plan_type	churn_rate_pct
0	Basic	60.00
1	Premium	14.29
2	Standard	22.22

```
# churn by state + sum(revenue) & count of users  
# churn by subscription type + sum(revenue) & count of users  
#df.head()  
churn_by_state = (df.groupby('state')['churn_flag'].mean().mul(100).round(2).reset_index(name='churn_rate_pct'  
print(churn_by_state)
```

	state	churn_rate_pct
0	Delhi	25.00
1	Karnataka	100.00
2	Kathmandu	0.00
3	Maharashtra	0.00
4	Meghalaya	66.67
5	Nagaland	0.00
6	Rajasthan	0.00
7	Telangana	50.00
8	Uttar Pradesh	0.00

```
# ARPU: Avg Revenue Per User  
arpu = df['monthly_charges'].mean()  
print('ARPU =', round(arpu, 2))
```

ARPU = 18.85

```
# Avg Customer tenure  
# count of days users has used our service: cancellation date else current date  
today = pd.Timestamp.today()  
  
df['tenure_days'] = np.where(  
    df['cancellation_date'].notna(),  
    (df['cancellation_date'] - df['subscription_start_date']).dt.days,  
    (today - df['subscription_start_date']).dt.days  
)  
  
avg_tenure = df['tenure_days'].mean()  
print("Avg Tenure (Days) = ", round(avg_tenure))
```

Avg Tenure (Days) = 1523

```
df.columns
```

```
Index(['customerid', 'subscription_start_date', 'subscription_type',
      'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
      'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
      'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
      'complaint_date', 'escalations', 'csat_score', 'complaint_count',
      'tenure_days'],
      dtype='str')
```

```
revenue_at_risk = df.loc[df['churn_flag']==1, 'monthly_charges'].sum()
print("Revenue at Risk (Rs 'K') =", revenue_at_risk)
```

Revenue at Risk (Rs 'K') = 73.94

```
# Escalation Rate
escalation_rate = (df['escalations'] == 'Y').mean()*100
print("Escalation Rate =", round(escalation_rate, 2), "%")
```

Escalation Rate = 19.05 %

```
#Avg Complaint Per User
avg_complaints = df['complaint_count'].sum() / df['customerid'].nunique()
print("Avg Complaints Per User = ", round(avg_complaints, 2))
```

Avg Complaints Per User = 0.43

```
# Correlation Escalation vs Churn
```

```
df['escalations'] = np.where(df['escalations'] == 'Y', 1, 0) # encoding string to int type
corr_df = df[['escalations', 'churn_flag']].dropna()
```

```
correlation = corr_df['escalations'].corr(df['churn_flag'])
print("Correlation between escalation vs churn is = ", round(correlation, 2))
```

Correlation between escalation vs churn is = 0.77

```
# Churn risk: create a column using existing col - churn risk
```

```
conditions = [
    (df['churn_score'] < 50),
    (df['churn_score'] >= 50) & (df['churn_score'] < 70),
    (df['churn_score'] >= 70)
]
choices = ['low', 'med', 'high']

df['churn_risk'] = np.select(conditions, choices, default = 'unknown')
```

7. Data Visualization

Matplotlib

```
df_visual = df.copy()
```

```
# Monthly Churn Trend (Time Series KPI)
```

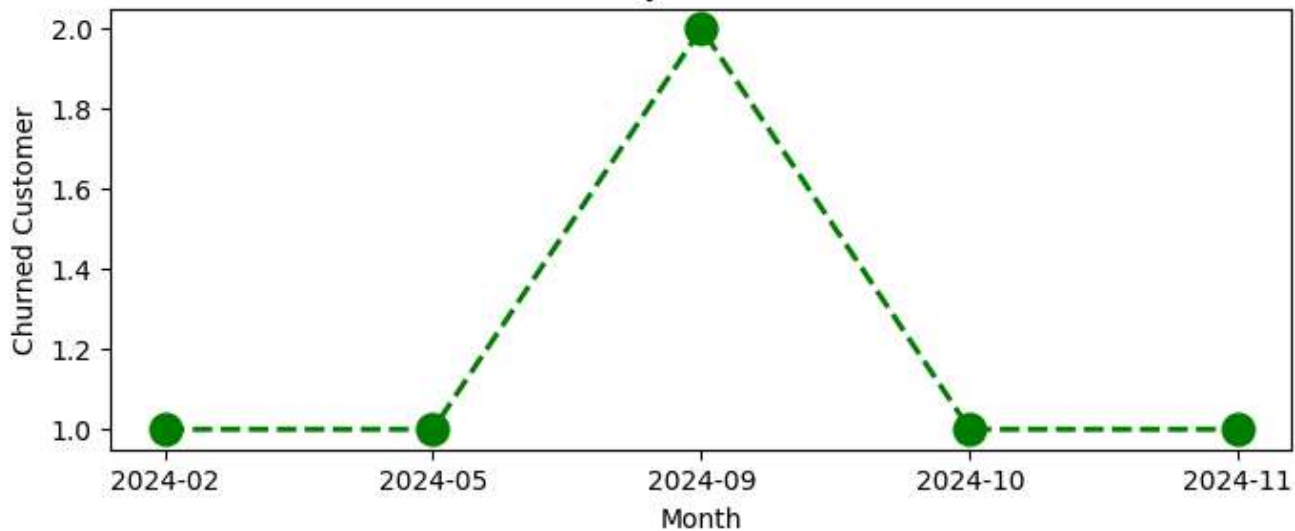
```
df_visual['cancellation_month'] = df_visual['cancellation_date'].dt.to_period('M')
```

```
churn_trend = df_visual[df_visual['churn_flag'] == 1].groupby('cancellation_month').size()
```

```
plt.figure(figsize=(8,3))
plt.plot(churn_trend.index.astype(str), churn_trend.values, color='green', marker='o', linestyle='dashed', lin

plt.title("Monthly Churn Trend")
plt.xlabel("Month")
plt.ylabel("Churned Customer")
plt.show()
```

Monthly Churn Trend

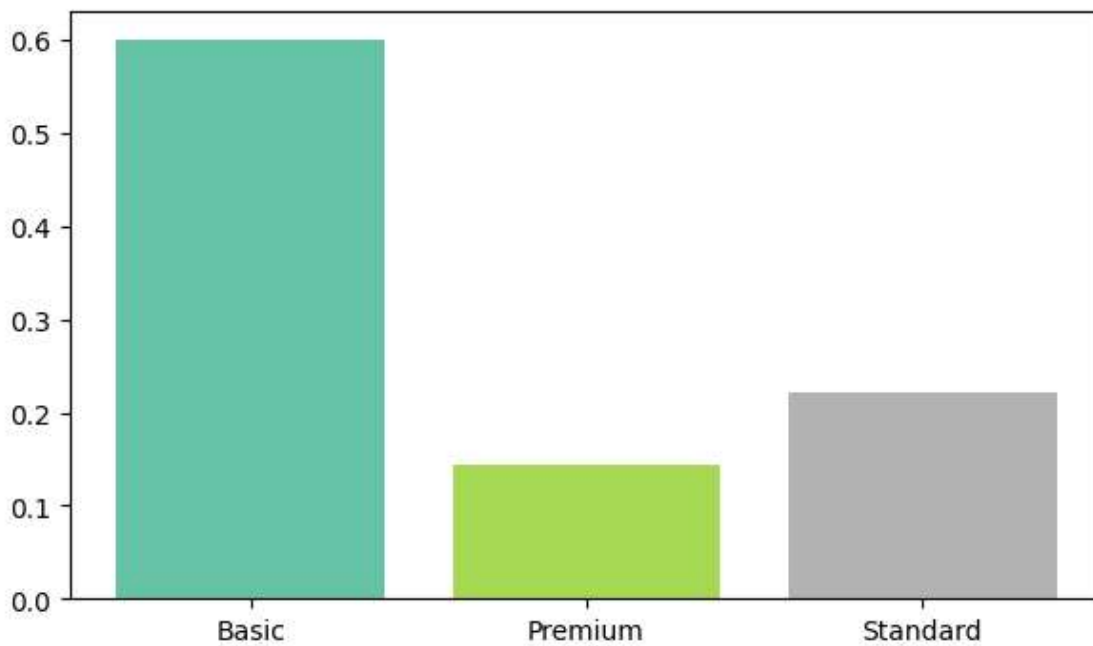


```
# churn by plan type
churn_plan = df_visual.groupby('plan_type')['churn_flag'].mean()

# colors = ['yellow', 'purple']
colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))

plt.figure(figsize=(7,4))

plt.bar(churn_plan.index, churn_plan.values, color = colors)
plt.show()
```

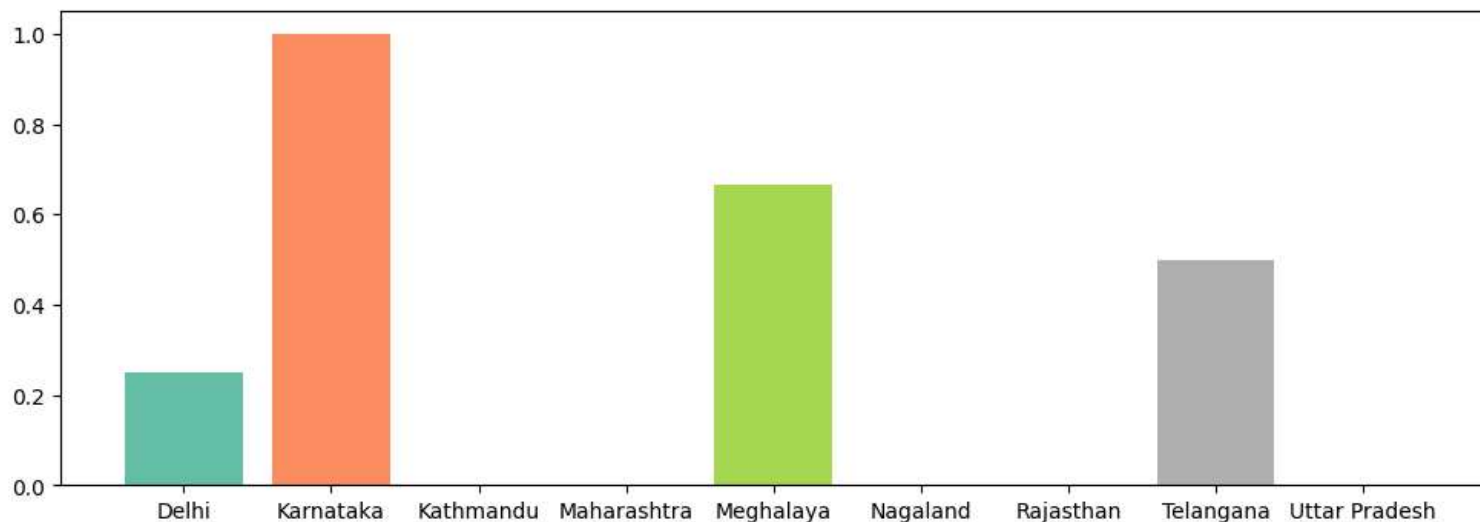


```
# churn by states
churn_plan = df_visual.groupby('state')['churn_flag'].mean()

# colors = ['yellow', 'purple']
colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))

plt.figure(figsize=(12,4))

plt.bar(churn_plan.index, churn_plan.values, color = colors)
plt.show()
```

```
# Visualization using Seaborn
```

```
# encoding - convert str to numeric so that we can find corr between features
df_visual.columns
```

```
Index(['customerid', 'subscription_start_date', 'subscription_type',
      'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
      'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
      'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
      'complaint_date', 'escalations', 'csat_score', 'complaint_count',
      'tenure_days', 'churn_risk', 'cancellation_month'],
      dtype='str')
```

```
df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_risk', 'escalations
```

```
order_mappings = {
    'plan_type' : ['Basic', 'Standard', 'Premium'],
    'contract_type' : ['Monthly', 'Annual'],
    'churn_risk' : ['low', 'med', 'high']
}
```

```
for col, order in order_mappings.items():
    df_encoded[col] = pd.Categorical(df_encoded[col].astype('category'), categories=order, ordered=True).codes
```

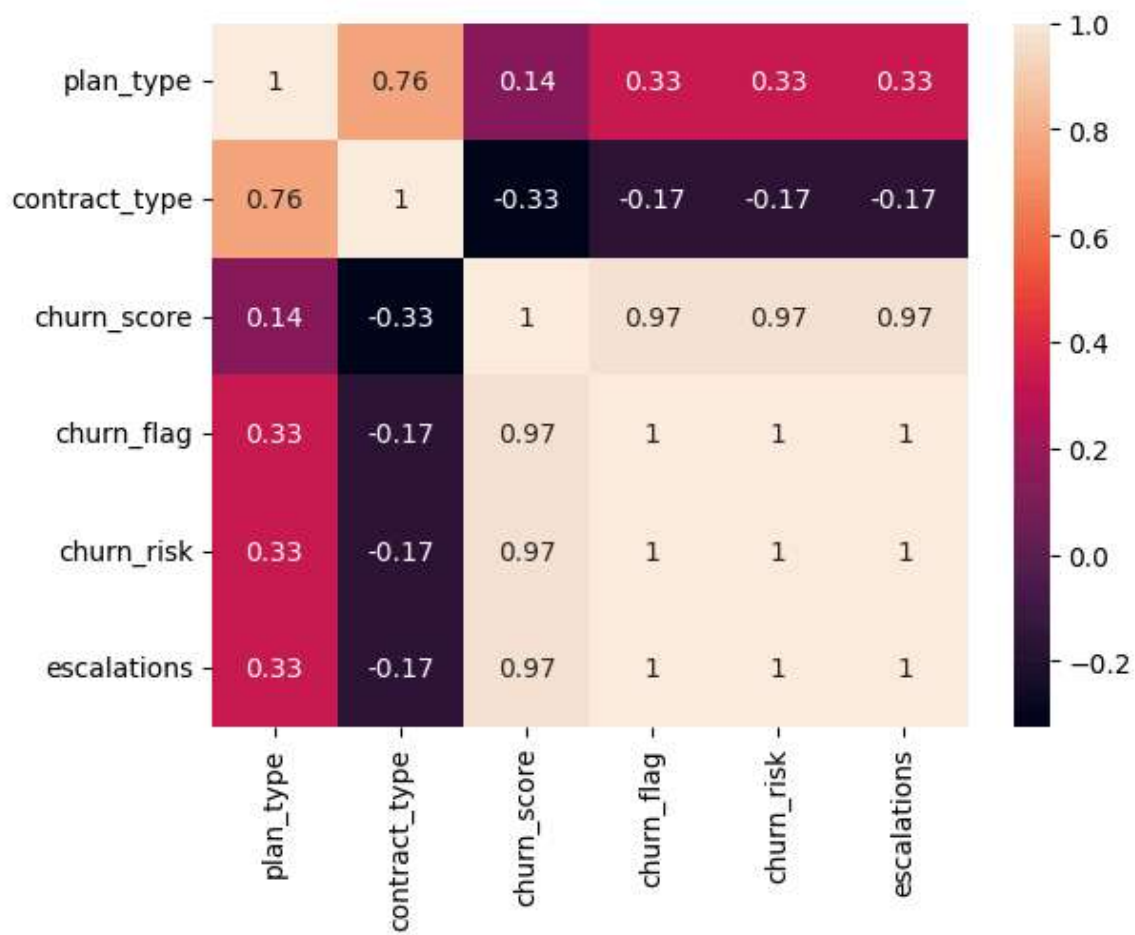
```
df_encoded.head()
```

	plan_type	contract_type	churn_score	churn_flag	churn_risk	escalations
0	1	1	12	0	0	0
1	2	1	91	1	2	1
2	0	0	34	0	0	0
3	2	1	8	0	0	0
4	1	0	88	1	2	1

```
# Heatmap (correlation matrix)
```

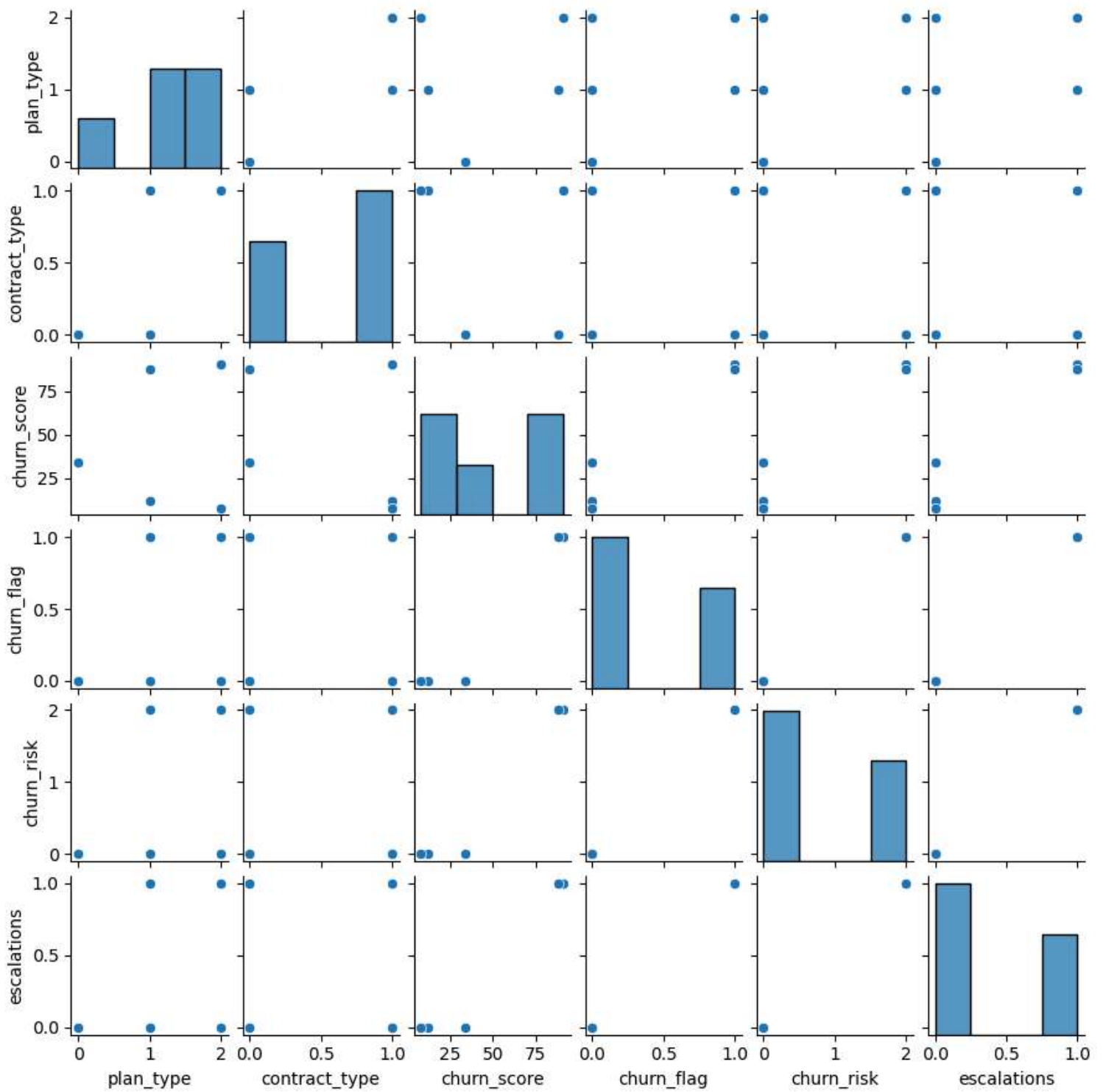
```
sns.heatmap(df_encoded.corr(), annot=True)
```

<Axes: >



```
# pairplot  
sns.pairplot(df_encoded, height=1.5, aspect=1)
```

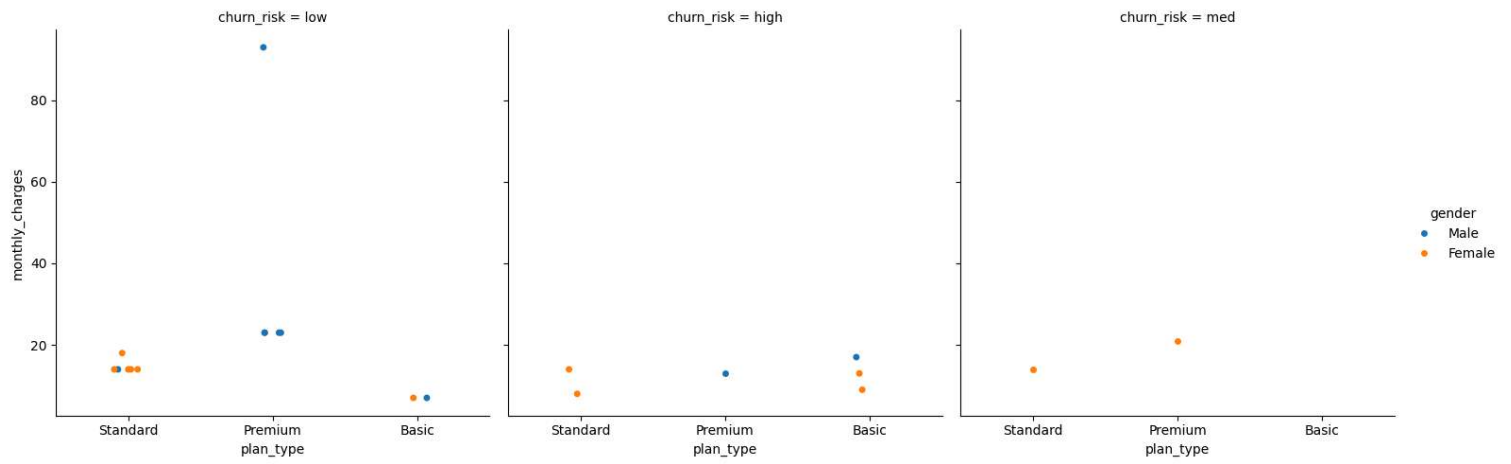
```
<seaborn.axisgrid.PairGrid at 0x1d244d6f710>
```



```
# catplt/Facegrid plot
```

```
sns.catplot(data=df_visual, x='plan_type', y='monthly_charges', hue='gender', col='churn_risk',)
```

```
<seaborn.axisgrid.FacetGrid at 0x1d24496d5e0>
```



pivot Table

```
pd.pivot_table(  
    df_visual,  
    index = 'plan_type',  
    values = ['monthly_charges', 'customerid', 'churn_flag'],  
    aggfunc = {  
        'monthly_charges': 'sum',  
        'customerid' : 'nunique',  
        'churn_flag' : 'mean'  
    }  
)
```

	churn_flag	customerid	monthly_charges
plan_type			
Basic	0.600000	5	52.95
Premium	0.142857	7	218.93
Standard	0.222222	9	123.91

8. Key Insights and Recommendations

Key Insights

- 1: Overall Customer Churn:** The analysis found an overall churn rate of 28.6%, while the retention rate was 71.4%.
- 2: Churn by Subscription Plan:** The Basic plan had the highest observed churn rate in this dataset, indicating that Basic-plan customers may require further investigation and targeted retention efforts.
- 3: Churn by Location:** Karnataka had the highest observed churn rate among the analyzed states, making it an important area for further investigation.
- 4: Revenue at Risk:** The analysis calculated approximately ₹39 of monthly revenue associated with churned customers, highlighting the potential financial impact of customer churn.
- 5: Support Escalations:** The analysis showed a strong positive correlation of approximately 0.77 between support escalations and churn. This suggests that customers with escalated support issues may be more likely to churn.
- 6: Retention:** The retention rate of 71.4% shows that most customers remained active, although the churned customer segment represents an important opportunity for retention improvement.

Business Recommendations

1: **Investigate Basic-plan churn:** Analyze whether pricing, features, service quality, or customer expectations are contributing to the higher observed churn among Basic-plan customers.

2: **Focus on high-churn locations:** Investigate customer experience, support complaints, service quality, and possible competitive factors in Karnataka.

3: **Prioritize customers with support escalations:** Create an early-warning or follow-up process for customers whose issues have been escalated.

4: **Target high-risk customers:** Use churn scores and churn-risk categories to identify customers who may need proactive retention efforts.

5: **Analyze cancellation reasons:** Review cancellation reasons such as switching to competitors or pricing concerns to identify recurring patterns.